In the SeekingAd case, there is no owner field in the form, and as it is a required field on the ORM object, attempting to save without it results in an exception. Using commit=False, you get an instance of the object before it is saved, so you can modify it before storing it in the database. Add the form handling view code from the following listing to RiffMates/content/views.py.

Listing 7.10 View for displaying and submitting SeekingAdForm

```
# RiffMates/content/views.py
from django.contrib.auth.decorators import login_required
...
from content.forms import CommentForm, SeekingAdForm
from content.models import MusicianBandChoice, SeekingAd

@login_required
def seeking_ad(request):
    if request.method == 'GET':
        form = SeekingAdForm()

    else: # POST
        form = SeekingAdForm(request.POST)

        if form.is_valid():
            ad = form.save(commit=False)
            ad.owner = request.user
            ad.save()

            return redirect("list_ads")

    # Was a GET, or Form was not valid
    data = {
        "form": form,
    }

    return render(request, "seeking_ad.html", data)
```

Ads belong to users and enforce that they are logged in.

The GET mode displays an empty form.

The POST mode populates the form based on the user's submitted data.

Verify the form's contents.

Save the form, creating a SeekingAd object without saving the model.

Save the SeekingAd model.

Add the user's account object to the SeekingAd object.

Redirect to the classified listing page.

Handle the GET and invalid form cases by displaying the form.

To go along with your new view, you'll need a template. Add the code from the following listing to RiffMates/templates/seeking_ad.html.

Listing 7.11 Template for the classified ad submission form

```
<!-- RiffMates/templates/seeking_ad.html -->
{% extends "base.html" %}

{% block content %}
<h1>Seeking Ad</h1>

<form method="post">
  {% csrf_token %}
  {{form.as_p}}
```